

LLM 运行机制：Token、上下文窗口与采样参数怎么影响输出

在探讨 RAG、Agent 工作流、MCP 协议这些高深概念之前，我想先聊聊一个让小 G 踩过不少坑的基础问题：明明设置了温度为 0，结构化输出还是崩；往模型里塞了一堆文档，它好像直接失忆，关键指令全当空气。说到底，还是底层原理没搞清楚。

万丈高楼平地起。这篇文章就是来填这个坑的。我们暂时把顶层架构放一放，回到 LLM 的基本面上来：Token 怎么算、上下文窗口怎么管、采样参数怎么调。

本文会沿着一条主线展开：先看模型为什么被 Token 和上下文窗口限制，再看采样参数如何影响输出稳定性，最后落到 Token 预算和参数配置建议。

具体会讲清楚：

1. 大模型（LLM）到底在做什么？
2. Token 是什么？为什么中文和英文的 Token 消耗差很多？
3. 上下文窗口是什么？为什么会有上限？
4. Temperature、Top-p、Top-k 这些采样参数怎么影响输出？
5. Token 预算怎么做？

Token 和上下文为什么决定成本与效果？

当你在输入法里打“今天天气真”，它会自动建议“好”——大模型做的事情本质上一样。只不过它看的不是前面几个字，而是前面几千甚至几十万个字。每次只“补”一个 Token（文本碎片），然后把这个碎片加进上下文，再预测下一个，如此循环，直到生成完整回答。

这个过程叫做自回归生成（Autoregressive Generation）。

理解了自回归生成，后面所有概念都好办了：

- **Token**：模型每一步“补”的文本碎片。
- **上下文窗口**：模型在“补”之前能看到多少文本。
- **Temperature / Top-p**：模型选哪个候选碎片的策略。
- **Max Tokens**：允许模型最多“补”多少步。

你可以把 Token 理解为“模型的阅读单位”。我们人类读中文是一个字一个字地看，读英文是一个词一个词地看。但模型既不按字、也不按词——它用一套自己的“拆字规则”（叫 Tokenizer）把文本切成大小不等的碎片，每个碎片就是一个 Token。

为什么不直接按字或按词切？因为模型需要在“词表大小”和“序列长度”之间取平衡：

- 每个汉字都是一个 Token，词表小、但序列长（模型要“补”更多步）。
- 每个词都是一个 Token，序列短、但词表会爆炸（中文词组太多了）。

所以实际用的是折中方案——**子词切分算法**（如 BPE、Unigram），高频词保留为整体，低频词拆成更小片段。

你可以把 Token 想象成乐高积木。常用的“积木块”比较大（比如“你好”可能是一个 Token），不常用的词会被拆成更小的基础块拼起来。

Token 不是“一个字”或“一个词”的严格等价物：

- 英文可能一个单词被拆成多个 Token。

- 中文可能一个词被拆成多个 Token，也可能多个字合并成一个 Token（取决于词频与词表）。

工程上通常用**经验估算**做容量规划，用**实际 API 返回的 usage**做精确计费与监控。

经验估算（仅用于粗略规划）：

- 英文：1 Token 大约对应 3~4 个字符（与文本类型相关）。
- 中文：1 Token 常见在 1~2 个汉字上下波动（与混排比例强相关）。

DeepSeek 官方数据：1 个英文字符约消耗 0.3 Token，1 个中文字符约消耗 0.6 Token。换算过来，1 个 Token 约等于 3.3 个英文字符或 1.7 个中文字符，与上述经验值吻合。

成本趋势提示：Token 成本与 Tokenizer 版本强相关。早期模型（如 GPT-3.5）中文压缩率较低（约 1 字 1.5~2 Token）。GPT-4o 使用 o200k_base Tokenizer（词表约 20 万），对中文压缩率有进一步提升；Qwen2.5 词表约 15 万，对中文常用词也有优化。实测数据因文本类型而异：新闻类约 1.5 字/Token，技术文档约 1.2 字/Token。

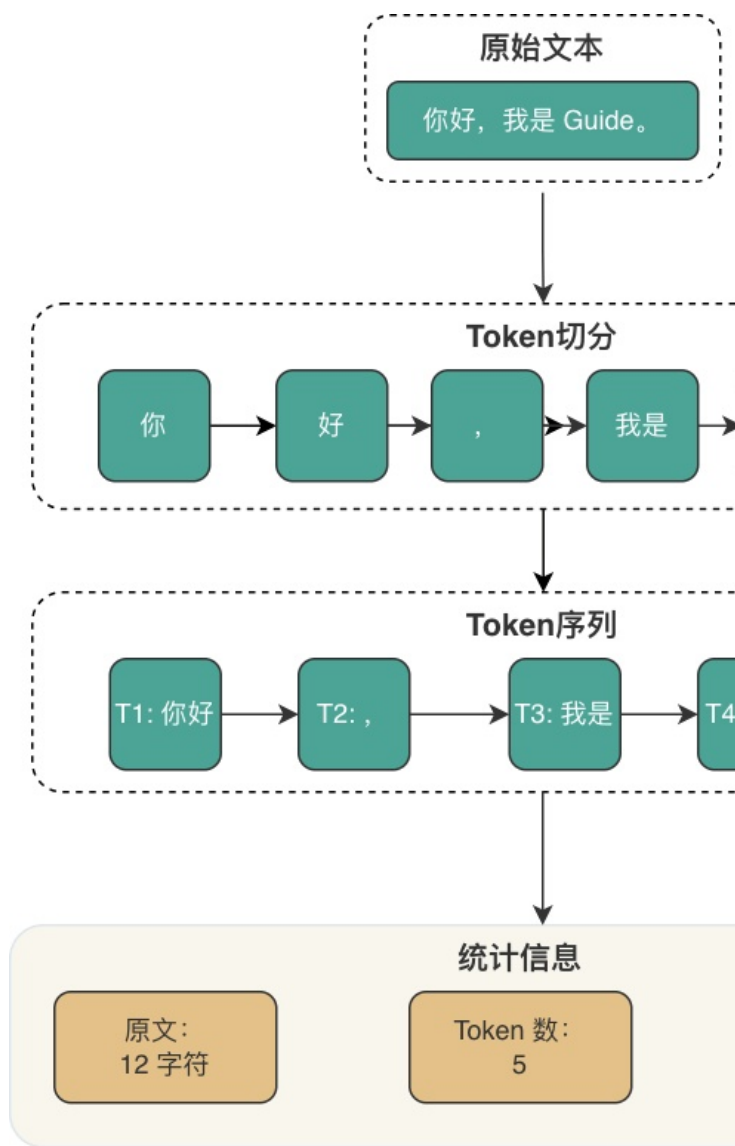
“趋近 1 字 1 Token”只适用于高频词汇，别拿它当成本估算基准。做预算前查一下当前模型版本的官方 Tokenizer 演示。

Token 划分直接影响模型理解能力。中文分词歧义和生僻字/低频专业术语的切分粒度，都会影响语义理解效果。

Token 化过程示例：

- 原文：你好，我是小 G。
- 切分：[你好] [,] [我是] [小 G] [。]

- 统计：原文 9 字符 → Token 数 5 个 → 压缩比约 1.8 倍



Token 化过程示例

注意：实际 Token 切分由模型供应商的 Tokenizer 实现，不同供应商对相同文本可能产生不同的 Token 序列。

OpenAI 官方网页端 Tokenizer 工具：[OpenAI Tokenizer](#)

特殊 Token：除了文本内容对应的 Token，模型内部还会使用一些特殊标记，这些也会计入 Token 总数：

特殊 Token	用途	示例
BOS (Beginning of Sequence)	标记序列开始	<s>
EOS (End of Sequence)	标记序列结束	</s>
PAD (Padding)	批处理时填充短序列	<pad>
工具调用标记	Function Calling 边界	<tool_call/>

这些特殊 Token 通常对用户不可见，但会占用上下文窗口。精确计数时建议使用官方 Tokenizer 工具而非手动估算。

多模态输入的 Token 开销

GPT-4o、Claude 3.5、Gemini 等模型已支持图片输入。图片不是“零成本”的——它会被转换成一批 Token，同样占用上下文窗口。

粗略估算规则：

模型	图片 Token 计算方式	一张 1024×1024 图片约等于
GPT-4o	按分辨率 + 细节模式	低细节 ~85 tokens, 高细节 ~1105~765 tokens
Claude 3.5	固定 ~5 tokens (缩略图) 或 ~85 tokens (全图)	取决于图片模式
Gemini	按分辨率计算	~258 tokens (标准)

工程启示：

- 做多模态 RAG 时，要把图片 Token 也纳入预算。
- 批量处理图片时，注意首字延迟 (TTFT) 会显著增加。
- 如果只需要 OCR，考虑先用专门的 OCR 服务提取文字，再以纯文本形式送入模型。

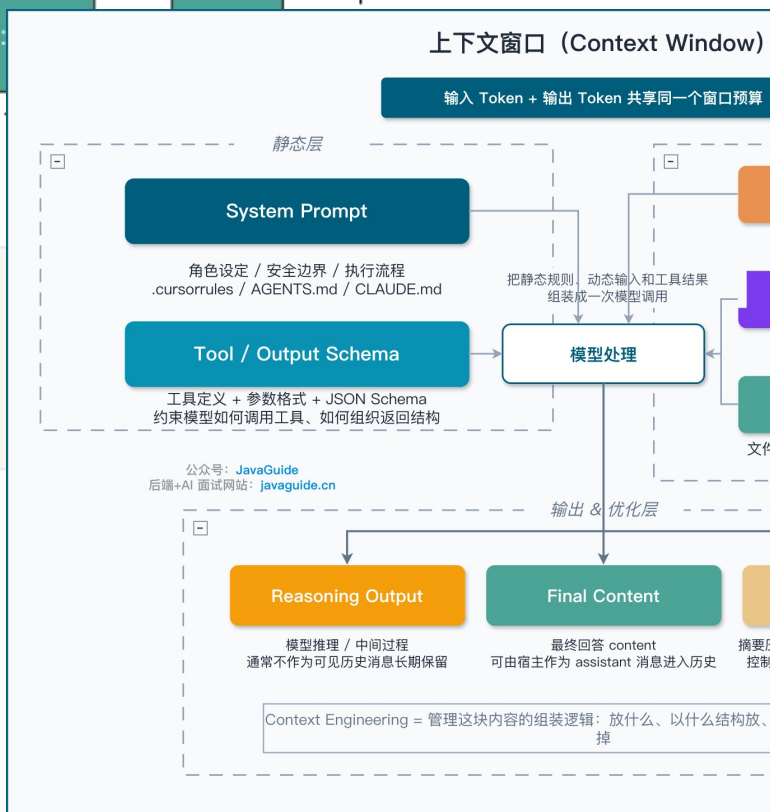
上下文窗口的容量边界

上下文窗口是 LLM 的“工作记忆” (Working Memory)。它决定了模型在任何时刻可以处理或“记住”的文本量 (以 Token 为单位)。

- 对话连续性：决定模型能进行多长的多轮对话而不遗忘早期细节。
- 单次处理能力：决定模型一次性能够处理的最大文档、代码库或数据样本。

“模型支持 128K/200K/1M” 指的是一次调用里能放进模型的总 Token 上限。大多数模型的上下文窗口包含输入与输出的总和，但部分供应商 (如 Google Gemini) 对输入和输出分别设限，使用前请查阅具体 API 文档。

上下文窗口往往被隐形成本占用：



上下文窗口 (Context Window) = LLM 的「工作记忆」

- System Prompt：调节模型行为的系统指令 (对用户隐藏，但占用窗口)。
- User Prompt：业务数据与指令。
- 多轮对话历史：过往的消息记录。
- RAG 检索片段：从外部知识库检索到的补充信息。
- 工具调用 Schema：函数定义与参数结构。
- 格式开销：特殊字符、换行符、Markdown 标记等。
- 模型生成的输出 Token：输出也占用上下文窗口。

因此，你真正能塞进 Prompt 的“有效业务内容”往往远小于标称上限。

注意：上下文窗口 (Context Window) 最大生成长度。许多模型支持 128K 甚至 1M 输入，但单次输出上限因 API 而异。OpenAI Chat Completions API 使用 `max_tokens`

参数 (GPT-4o 最大 16K 输出) , 部分新模型支持 `max_completion_tokens` (如 o1 系列), DeepSeek V3 最大输出 8K。使用前需查阅具体模型的 API 文档。

思维链模式的多轮对话处理: 思维链模型 (如 DeepSeek-R1) 的 `reasoning_content` (思考过程) 通常不会被自动包含在下一轮对话的上下文中, 只有 `content` (最终回答) 会参与后续对话。

这意味着:

- 无需为思考过程额外占用上下文窗口。
- 如果后续对话需要参考之前的推理过程, 需要手动将 `reasoning_content` 拼接到消息历史中。
- 部分供应商的 SDK 会自动处理这一差异, 建议查阅具体文档确认。

长上下文背后的计算约束

上下文窗口并非越大越好, 它受限于 Transformer 架构的自注意力机制 (Self-Attention):

- 计算成本平方级增长: 计算需求与序列长度呈平方级关系 ($O(N^2)$)。输入 Token 翻倍, 处理能力需求可能变为 4 倍。
- 推理延迟增加: 上下文变长后, 模型生成每个新 Token 时需要关注的历史 Token 变多, 首字延迟 TTFT 会显著增加。
- 安全风险增加: 更长的上下文意味着更大的攻击面。

工程优化手段: FlashAttention、GQA/MQA、Sliding Window Attention、Ring Attention 等技术已显著降低长上下文的计算和显存开销。但 $O(N^2)$ 的理论复杂度仍是上限扩展的根本瓶颈。

上下文溢出的真实表现

当上下文接近上限或内容过长时, 常见现象包括:

- 模型忽略早期约束: System Prompt 里要求 “必须输出 JSON”, 但因距离生成点太远, 注意力不足导致被忽略。
- “中间丢失” 现象: 即使在 1M 窗口模型中, 模型对开头和结尾的信息最敏感, 对中间部分的信息召回率显著下降。
- 回答漂移: 前半段还围绕问题, 后半段开始总结/扩写/跑题。
- RAG 失效: 检索文档过多, 关键信息被稀释; 或被截断导致证据链断裂。
- 成本与延迟激增: 1M 上下文会导致 TTFT 显著增加, 且 Token 成本呈线性增长。

输入 Token 与输出 Token 的计费差异

大多数供应商对输入 Token 和输出 Token 采用不同的计费标准, 通常输出价格是输入的 2~4 倍:

模型	输入价格 (/1M Tokens)	输出价格 (/1M Tokens)	输出/输入比
GPT-4o	\$2.50	\$10.00	4x
Claude 3.5 Sonnet	\$3.00	\$15.00	5x
DeepSeek V3	¥0.5	¥2.0	4x
DeepSeek-R1	¥4.0	¥16.0	4x

工程启示:

- 长 Prompt + 短输出 = 更经济的调用方式。
- RAG 场景要控制检索片段数量, 避免输入 Token 激增。
- 思维链模型的 reasoning tokens 通常按输出价格计费, 成本更高。

Prompt Caching 的省钱逻辑

当请求中存在大量重复的固定前缀 (如 System Prompt、长 RAG Context), 可以用 **Prompt Caching** 显著降低成本。

原理: 供应商会缓存请求中 “可复用的前缀部分”。下次请求如果前缀相同, 这部分就不重新计费, 只收 “缓存读取” 的费用 (通常是正常价格的 10%~50%)。

典型适用场景:

- 多轮对话 (System Prompt + 历史 Message 不变)。
- RAG 应用 (检索片段重复率高)。
- 批量评估 (同一份 System Prompt, 不同的简历/文章)。

各供应商支持情况:

供应商	功能名称	缓存时长	缓存命中折扣
OpenAI	Prompt Caching	5~10 分钟	输入价格约 50
Anthropic	Prompt Caching	5 分钟	输入价格约 10
DeepSeek	Context Caching	10~30 分钟	输入价格约 25

工程建议:

1. 把不变的内容放前面 (System Prompt、工具定义、RAG Context), 把变化的内容放后面 (User Prompt)。
2. 监控 `cache_read_tokens` 和 `cache_creation_tokens` 指标, 验证缓存命中率。
3. 批量任务尽量在缓存时间窗口内完成。

一次调用的 Token 预算公式

把 “上下文窗口” 当成一个固定容量的桶, 下图展示了一个典型调用的 Token 预算分配:

此分配仅为示意, 实际比例需根据业务场景动态调整。

最实用的预算方式是:

window input_tokens + max_output_tokens

对于思维链模型, 公式应调整为:

window input_tokens + reasoning_tokens + max_output_tokens

其中 reasoning_tokens (思考链 Token 数) 难以精确预估, 建议按 max_output_tokens 的 2~3 倍预留。

其中 input_tokens 至少包含:

- system prompt (含 schema / 工具定义)
- user prompt (含变量替换后的实际文本)
- 历史消息 (多轮对话时)
- RAG context (如果拼进来了)

工程上建议反过来做预算 (因为输出经常更可控):

1. 先定 max_output_tokens (结构化输出通常不需要很长)。
 2. 再为输入预留安全边际 (例如再留 10%~20% 给供应商额外开销)。
 3. 超预算时, 用可解释的策略 “减输入” 而不是 “赌模型会自我约束”:
- 优先减少 RAG 的 Top-K 或做片段去重。
 - 对长字段做摘要/截断 (如简历、长回答)。
 - 多段任务拆成多次调用 (分批评估、两阶段生成)。

采样参数如何影响输出稳定性？

从 logits 到概率采样

模型每一步会给词表中每个候选 Token 打一个分数（内部叫 **logits**），分数越高说明模型越觉得这个词应该出现在这里。举个例子，假设模型正在补全“今天天气真 ____”，它可能给出这样的分数：

候选 Token	原始分数 (logit)
好	5.0
不错	3.2
棒	2.1
糟糕	0.5
紫色	-8.0

但原始分数不是概率——需要经过一次数学变换（**softmax**）才能变成每个候选被选中的概率。变换后大致是：

候选 Token	概率
好	62
不错	20
棒	10
糟糕	5
紫色	0

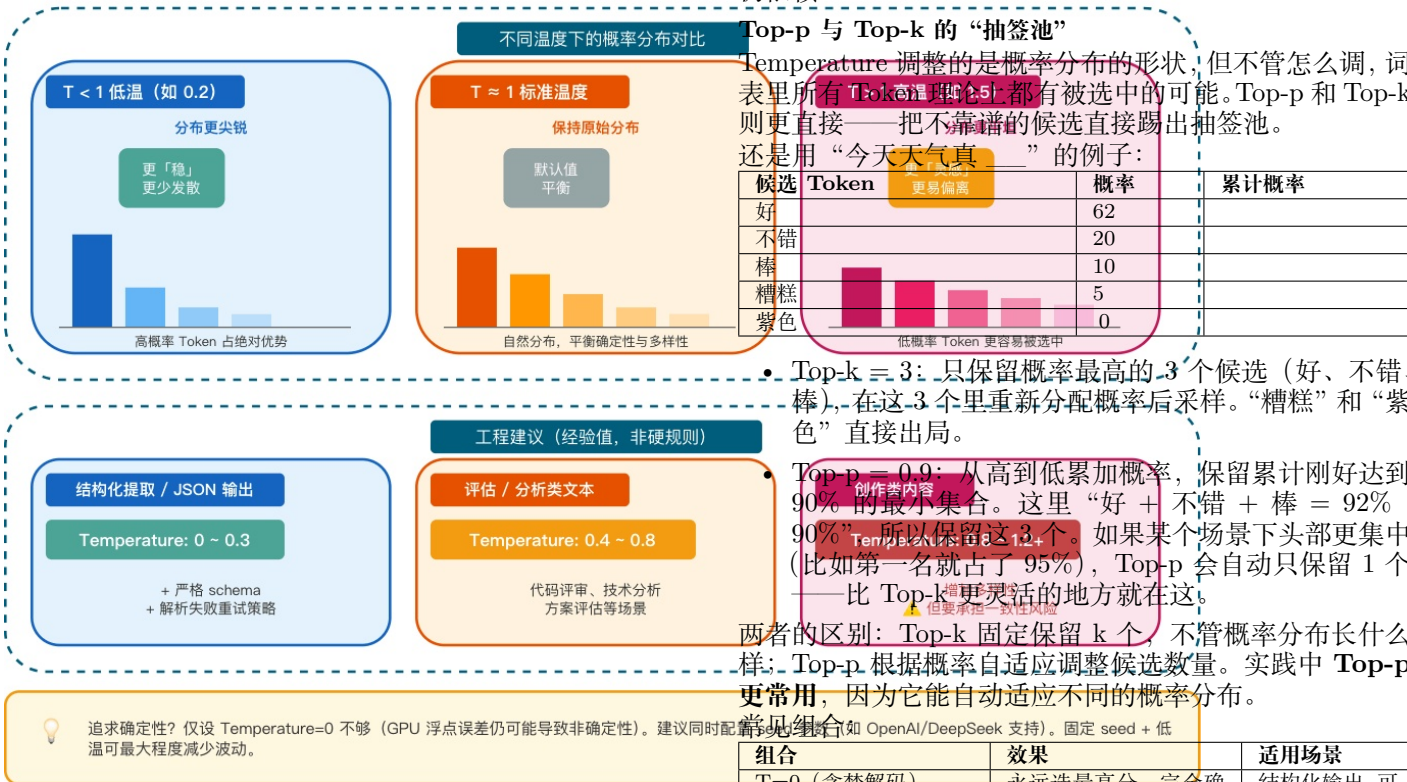
最后，模型按这个概率分布“抽签”（采样），决定输出哪个 Token。

解码参数（Temperature、Top-p、Top-k 等）就是在这个“打分 → 概率 → 抽签”的过程中施加控制：

- Temperature：调整概率分布的“形状”，让高分选项更突出，或者让各选项更均匀。
- Top-p / Top-k：直接砍掉不靠谱的候选项，缩小“抽签池”。
- Penalty 系列：对已经出现过的词降分，防止“复读机”。

Temperature 的“冒险程度”

Temperature 参数：控制模型输出的随机性



Temperature 参数：控制模型输出的随机性

Temperature 的工作原理很简单：在 softmax 之前，先把所有分数除以温度值 T。

$$p(t) = \text{softmax}(z_t / T)$$

- T = 1：保持原始分布。
- T < 1：分布更尖锐，更倾向选择高概率 Token（更“稳”）
- T > 1：分布更平坦，低概率 Token 更容易被采样到（更“野”）

还是用“今天天气真 ____”的例子：

- T = 0.2 (低温)：分数差距被放大（都除以 0.2，等于乘以 5），原本就领先的“好”概率飙升到 ~98%，几乎每次都选它。
- T = 1.0 (默认温度)：保持原始分布不变，“好”62%、“不错”20%... 按正常概率采样。
- T = 1.5 (高温)：分数差距被缩小（都除以 1.5），“好”概率降到 ~35%，“棒”、“不错”甚至“糟糕”都有更大机会被选中。

温度越低，输出越确定；温度越高，输出越随机。

工程建议（经验值，非硬规则）：

场景	推荐温度	说明
结构化提取 / JSON 输出	0 ~ 0.3	配合严格 schema + 解析失败重试策略
评估 / 分析 / 代码评审	0.4 ~ 0.8	平衡确定性与表达多样性
创作类内容 (文案、头脑风暴)	0.8 ~ 1.2+	增加多样性, 但要承担格式一致性风险

追求确定性? 若需单元测试幂等或结果复现，仅设 Temperature=0 不够（GPU 浮点误差仍可能导致非确定性）。建议同时配置 **seed 参数**（如 OpenAI/DeepSeek 支持）。

即使配置 seed，以下情况仍可能导致结果不一致：

- 模型版本更新（底层权重变化）。
- 跨区域调用（不同集群可能部署不同版本）。

Top-p 采样（即使 T=0，若 Top-p<1 仍有随机性）。

建议在 CI/CD 中仅将 LLM 调用用于冒烟测试，核心逻辑仍依赖 Mock。

Top-p 与 Top-k 的“抽签池”

Temperature 调整的是概率分布的形状，但不管怎么调，词表里所有 Token 理论上都有被选中的可能。Top-p 和 Top-k 则更直接——把不靠谱的候选直接踢出抽签池。

还是用“今天天气真 ____”的例子：

候选 Token	概率	累计概率
好	62	
不错	20	
棒	10	
糟糕	5	
紫色	0	

• Top-k = 3：只保留概率最高的 3 个候选（好、不错、棒），在这 3 个里重新分配概率后采样。“糟糕”和“紫色”直接出局。

• Top-p = 0.9：从高到低累加概率，保留累计刚好达到 90% 的最小集合。这里“好 + 不错 + 棒 = 92%”，所以保留这 3 个。如果某个场景下头部更集中（比如第一名就占了 95%），Top-p 会自动只保留 1 个——比 Top-k 更灵活的地方就在这。

两者的区别：Top-k 固定保留 k 个，不管概率分布长什么样；Top-p 根据概率自适应调整候选数量。实践中 **Top-p 更常用**，因为它能自动适应不同的概率分布。

组合	效果	适用场景
T=0 (贪婪解码)	永远选最高分，完全确定	结构化输出、可复现场景
低温 + Top-p=0.9	相对稳定，但允许措辞上有些变化	分析报告、摘要
中高温 + Top-p=0.95	多样性较高，但排除了极端离谱选项	创意写作、对话

注意：贪婪解码虽然最稳定，但可能更容易陷入重复循环。

停止条件与截断风险

工程上需要意识到两点：

- **Max Tokens 是硬上限**：到上限会被强制截断，模型正写到一半也会被“掐断”。常见后果：JSON 缺右括号、列表缺最后几项、句子写了一半。
- **Stop Sequences (停止词) 是软切断**：可以指定一些字符串（如“\n\n”或“ ”“ ”“ ”），模型生成到这些内容时会自动停止。但如果 stop 设计不当，可能提前截断关键字段。

结构化输出场景要把“截断风险”当成一类失败路径来设计缓解策略。

思维链模式的 Token 计算差异：对于支持思维链的模型（如 DeepSeek-R1），max_tokens 通常包含思考过程 + 最终回答两部分。例如设置 max_tokens=8192，模型可能在思考链上消耗 5000 tokens，最终回答只剩 3192 tokens 的预算。

不同供应商的默认值和上限差异较大：DeepSeek-R1 默认 32K、最大 64K；OpenAI o1 系列的输出上限也高于普通模型。使用前务必查阅具体模型的 API 文档。

Penalty 与复读问题

可能遇到过模型反复输出同一句话，或者在长回答里不断重复相同观点。Penalty 参数用来缓解这类问题，它们在解码时降低已出现 Token 的分数：

参数	作用	通俗理解
Repetition Penalty	降低所有已出现 Token 的概率	“说过的词，再说就扣分”
Presence Penalty	只要 Token 出现过就扣分（不看次数）	“鼓励聊新话题”
Frequency Penalty	Token 出现次数越多扣分越重	“同一个词说了三遍？重罚”

工程陷阱：

- 结构化输出别乱加 Penalty：JSON 里字段名（如“name”、“score”）需要反复出现，加了 Repetition Penalty 可能把必须出现的字段名也“惩罚掉”，导致输出残缺。
- RAG 问答别加 Presence Penalty：它会鼓励模型“说点新东西”，反而降低对检索内容的忠实度，增加幻觉风险。

保守建议：如果不确定这些参数的精确语义（不同供应商定义可能不同），建议保持默认值。用低温 + 更强 Prompt 约束 + 更短输出来获得稳定性，比调 Penalty 更可控。

思维链模式的参数限制

部分模型（如 DeepSeek-R1、OpenAI o1）支持“思维链模式”，在生成最终回答前会先输出一段内部推理过程。这类模型有特殊的参数约束：

不支持的采样参数：思维链模式下，以下参数通常被忽略：

- temperature、top_p：采样控制参数。
- presence_penalty、frequency_penalty：惩罚参数。

原因：思维链模式的设计目标是让模型“自由思考”，采用模型内部固定的采样策略，用户传入的采样参数会被忽略。

工程建议：

- 调用思维链模型时，不要依赖上述参数控制输出风格。
- 若需要更稳定的输出格式，应通过 Prompt 约束而非采样参数。
- 关注模型返回的 reasoning_content 字段（思考过程）与 content 字段（最终回答）的区别。

流式输出与首字延迟

默认情况下，API 会等模型生成完所有内容后一次性返回。流式输出则是边生成边返回——模型每生成一个（或几个）Token，就立刻推送给客户端，用户更早看到内容开始出现。

核心价值：改善用户体验，降低首字延迟（TTFT, Time-To-First-Token）。

常见误解澄清：

- 流式输出更快——总耗时（E2E latency）不一定下降，模型生成的总 Token 量相同。
- 流式输出更省钱——Token 计费不变，仍然受限流/配额影响。
- 如果需要结构化输出（如 JSON），流式场景要考虑“半成品 JSON”在前端/网关层的处理。

Logprobs 与置信度排查

部分 API（如 OpenAI）支持返回每个生成 Token 的对数概率（logprobs），可以理解为模型对该 Token 的“确信程度”。logprob 越接近 0，模型越确信；值越小（如 -5.0），说明模型越“犹豫”。

工程应用场景：

- 置信度评估：提取“金额：1000”时，若对应 Token 的 logprob 很低，说明模型不太确定，可能需要人工复核。
- 异常检测：监控生产环境中模型输出的平均 logprob，若突然下降可能提示 Prompt 漂移或输入数据异常。
- 多候选对比：获取 Top-N 候选 Token 及其概率，用于纠错或二次排序。

注意事项：logprobs 会增加响应体积，且并非所有供应商都支持。使用前请查阅 API 文档。

采样参数配置建议

场景	Temperature	Top-p	Penalty	其他建议
JSON / 结构化输出	0 ~ 0.3	1.0	保持默认	配合 Strict Mode + 重试策略
代码评审 / 技术分析	0.4 ~ 0.7	0.9	保持默认	结合 CoT Prompt
多轮对话	0.6 ~ 0.8	0.9	适度开启	控制历史消息长度
创意写作 / 头脑风暴	0.8 ~ 1.2	0.95	按需开启	接受输出多样性，做好后处理
思维链模型	—（不支持）	—	—	通过 Prompt 控制，非采样参数

总结

回顾这篇扫盲内容，核心其实就是处理好三个维度的工程权衡：

1. **Token 是成本与性能的物理标尺**：它不仅决定计费账单和推理延迟，更决定模型对文本的理解粒度。做容量规划时，必须按 Token 算账，而不是按字数算账。
2. **上下文窗口是极其稀缺的资源**：哪怕模型宣称支持 1M 上下文，也不意味着可以毫无节制地堆砌数据。为 Prompt、RAG 检索片段、历史对话和输出预留做好严格的 Token 预算分配，是走向生产环境的必修课。
3. **采样参数是业务场景的调音台**：如果追求稳定的 JSON 输出，就果断压低 Temperature 并配合严格的 Schema；如果需要创意与头脑风暴，再适度放开 Temperature 和 Top-p。不要迷信默认参数，要根据业务的容错率来定制。

打好这层参数与原理的地基，再去 Agent 编排、RAG 检索或是 MCP 工具调用，你会发现那些高阶架构的本质，无非是在更好地调度这些底层 Token，更精准地管理这个上下文窗口。